

Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées Al Hoceima

COMPTE RENDU

TP N°02 : Deep Learning – MLP

Filière : TDIA 2 - S 4

Encadré par :

Pr. Mohamed CHERRADI

Réalisé par :

Ouhadi Chaimae

Introduction

Ce TP a pour but de mettre en pratique les concepts du Perceptron Multicouche (MLP). Le MLP est l'un des modèles fondamentaux du Deep Learning : c'est un réseau de neurones composé de plusieurs couches (entrée, cachées, sortie) où chaque neurone est connecté à tous les neurones de la couche suivante.

L'objectif est d'implémenter des MLP avec Keras/TensorFlow sur quatre jeux de données différents, chacun représentant un type de problème :

- **Exercice 1: MNIST:** Classification multi-classes de chiffres manuscrits (images 28×28 en niveaux de gris).
- **Exercice 2: CIFAR-10:** Classification multi-classes d'images en couleur (32×32×3) représentant 10 catégories d'objets.
- **Exercice 3: Breast Cancer:** Classification binaire de tumeurs mammaires (bénignes ou malignes) à partir de 30 attributs numériques.
- **Exercice 4: Boston Housing:** Régression pour prédire le prix médian des maisons à partir de 13 caractéristiques.

Exercice 01: Implémentation d'un Perceptron Multicouche (MLP) sur le jeu de données MNIST

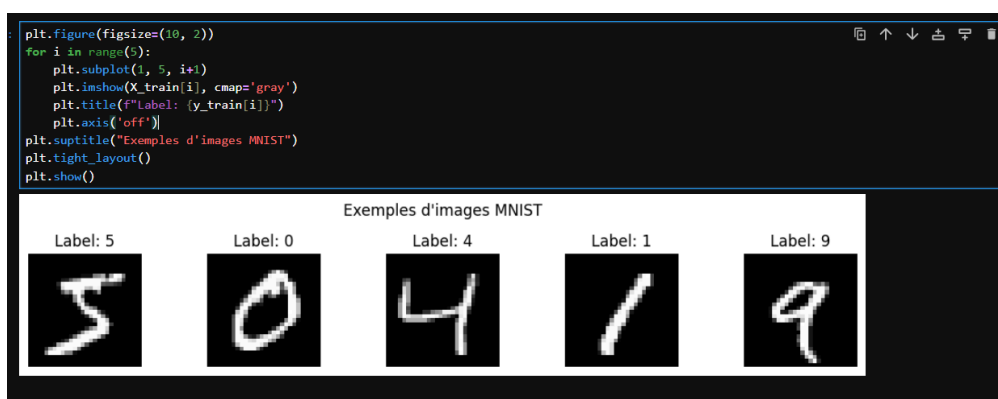
Après avoir importé les bibliothèques nécessaires (NumPy pour les calculs, Matplotlib pour la visualisation, et les modules Keras nécessaires pour construire et entraîner notre MLP), on passe au chargement des données MNIST:

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Taille des données d'entraînement :", X_train.shape)
print("Taille des données de test :", X_test.shape)
print("Étiquettes :", np.unique(y_train))

Taille des données d'entraînement : (60000, 28, 28)
Taille des données de test : (10000, 28, 28)
Étiquettes : [0 1 2 3 4 5 6 7 8 9]
```

On passe à la visualisation nous permet de vérifier que les images correspondent bien aux étiquettes associées.



Puis la partie du prétraitement des données:

- **Aplatissement:** Le MLP attend un vecteur 1D en entrée, pas une matrice 2D. Or nos images sont 28×28, donc nous sommes dans l'obligation de les transformer en un vecteur de 784 valeurs.
- **Normalisation:** On divise par 255 pour ramener les pixels entre 0 et 1. Cela accélère la convergence de l'optimiseur.
- **One-hot encoding:** La sortie Softmax produit 10 probabilités, donc les étiquettes doivent être en format one-hot. Par exemple, le chiffre 3 devient [0,0,0,1,0,0,0,0,0,0].

```

Prétraitement des données

# 28x28 en un vecteurs de 784
X_train = X_train.reshape(X_train.shape[0], 784)
X_test = X_test.reshape(X_test.shape[0], 784)

# Normalisation
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

print("Forme après aplatissement :", X_train.shape)

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

print("Forme des étiquettes après one-hot :", y_train.shape)
print("Exemple d'étiquette one-hot :", y_train[0])

Forme après aplatissement : (60000, 784)
Forme des étiquettes après one-hot : (60000, 10)
Exemple d'étiquette one-hot : [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
    
```

Le modèle Sequential empile les couches dans l'ordre. La première couche Dense reçoit des vecteurs de 784 éléments. ReLU est utilisé dans les couches cachées car il évite le problème du gradient qui disparaît. Softmax en sortie transforme les scores en probabilités pour les 10 classes.

```

model_mnist = Sequential()

# Couche cachée 1
model_mnist.add(Dense(128, activation='relu', input_shape=(784,)))

# Couche cachée 2
model_mnist.add(Dense(64, activation='relu'))

# Couche de sortie
model_mnist.add(Dense(10, activation='softmax'))

model_mnist.summary()
    
```

On compile le modèle avec Adam comme optimiseur et categorical_crossentropy comme fonction de perte qui est adaptée à la classification multi-classes. Le paramètre

validation_split=0.2 sépare automatiquement 20% des données d'entraînement pour la validation, ce qui permet de surveiller le surapprentissage.

```

Compilation

[7]: model_mnist.compile(
      optimizer='adam',
      loss='categorical_crossentropy',
      metrics=['accuracy'])
)

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Entraînement du modèle

[8]: history_mnist = model_mnist.fit(
      X_train, y_train,
      epochs=5,
      batch_size=64,
      validation_split=0.2
    )

Epoch 1/5
750/750 — 9s 8ms/step - accuracy: 0.9146 - loss: 0.2991 - val_accuracy: 0.9567 - val_loss: 0.1490
Epoch 2/5
750/750 — 6s 8ms/step - accuracy: 0.9647 - loss: 0.1177 - val_accuracy: 0.9654 - val_loss: 0.1142
Epoch 3/5
750/750 — 5s 7ms/step - accuracy: 0.9753 - loss: 0.0821 - val_accuracy: 0.9688 - val_loss: 0.1033
Epoch 4/5
750/750 — 5s 7ms/step - accuracy: 0.9819 - loss: 0.0589 - val_accuracy: 0.9711 - val_loss: 0.0971
Epoch 5/5
750/750 — 6s 7ms/step - accuracy: 0.9848 - loss: 0.0479 - val_accuracy: 0.9743 - val_loss: 0.0885

```

Pour l'évaluation du modèle:

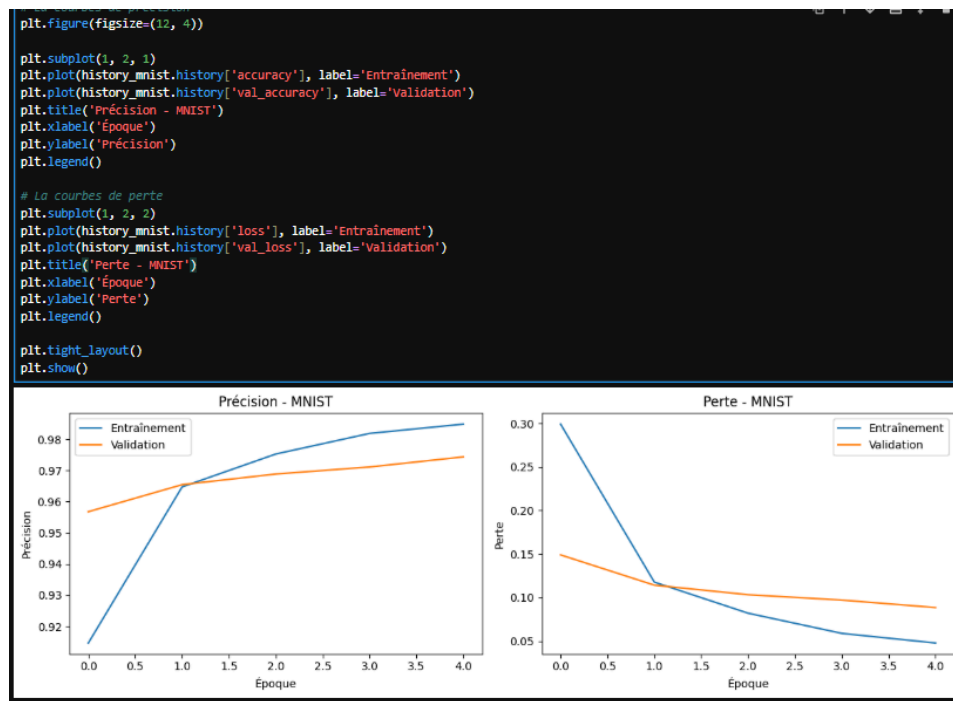
```

test_loss, test_accuracy = model_mnist.evaluate(X_test, y_test)
print(f"\nPerte sur le test : {test_loss:.4f}")
print(f"Précision sur le test : {test_accuracy:.4f}")

313/313 — 1s 4ms/step - accuracy: 0.9756 - loss: 0.0793

Perte sur le test : 0.0793
Précision sur le test : 0.9756

```



Les courbes montrent que la perte diminue et la précision augmente au fil des époques, tant pour l'entraînement que pour la validation.

Exercice 02: Implémentation d'un Perceptron Multicouche (MLP) sur le jeu de données CIFAR-10

CIFAR-10 est un jeu de données plus complexe que MNIST. Il contient 60 000 images en couleur de taille 32×32 pixels, réparties en 10 classes : avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau et camion. 50 000 images pour l'entraînement et 10 000 pour le test.

Chargement des données & visualisation d'exemple:

```
train = torchvision.datasets.CIFAR10(root='./data', train=True, download=True)
test = torchvision.datasets.CIFAR10(root='./data', train=False, download=True)

X_train_cifar = np.array(train.data)
y_train_cifar = np.array(train.targets).reshape(-1, 1)
X_test_cifar = np.array(test.data)
y_test_cifar = np.array(test.targets).reshape(-1, 1)

# Noms des classes
classes_cifar = ['avion', 'automobile', 'oiseau', 'chat', 'cerf',
                 'chien', 'grenouille', 'cheval', 'bateau', 'camion']

# Exploration
print("Taille des données d'entraînement :", X_train_cifar.shape)
print("Taille des données de test :", X_test_cifar.shape)
print("Nombre de classes :", len(classes_cifar))
```

```
ds_train, ds_test = tfds.load('cifar10', split=['train', 'test'], as_supervised=True, batch_size=-1)
X_train_cifar, y_train_cifar = tfds.as_numpy(ds_train)
X_test_cifar, y_test_cifar = tfds.as_numpy(ds_test)
```

```
# Visualisation de quelques exemples
plt.figure(figsize=(10, 2))
for i in range(5):
    plt.subplot(1, 5, i+1)
    plt.imshow(X_train_cifar[i])
    plt.title(classes_cifar[y_train_cifar[i][0]])
    plt.axis('off')
plt.suptitle("Exemples d'images CIFAR-10")
plt.tight_layout()
plt.show()
```

Prétraitement:

```
# Aplatir de 32x32x3 en vecteurs de 3072
X_train_cifar = X_train_cifar.reshape(X_train_cifar.shape[0], 3072)
X_test_cifar = X_test_cifar.reshape(X_test_cifar.shape[0], 3072)

# Normalisation
X_train_cifar = X_train_cifar.astype('float32') / 255.0
X_test_cifar = X_test_cifar.astype('float32') / 255.0

print("Forme après aplatissement :", X_train_cifar.shape)

y_train_cifar = to_categorical(y_train_cifar, 10)
y_test_cifar = to_categorical(y_test_cifar, 10)

print("Forme des étiquettes après one-hot :", y_train_cifar.shape)
```

Construction, compilation et entraînement du modèle:

```
model_cifar = Sequential()

# La couche cachée 1
model_cifar.add(Dense(512, activation='relu', input_shape=(3072,)))

# La couche cachée 2
model_cifar.add(Dense(256, activation='relu'))

# La couche de sortie
model_cifar.add(Dense(10, activation='softmax'))

model_cifar.summary()
```

```
model_cifar.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# Entraînement avec 10 époques, batch_size=64, validation_split=20%
history_cifar = model_cifar.fit(
    X_train_cifar, y_train_cifar,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)
```

Evaluation:

```
test_loss, test_accuracy = model_cifar.evaluate(X_test_cifar, y_test_cifar)
print(f"\nPerte sur le test : {test_loss:.4f}")
print(f"Précision sur le test : {test_accuracy:.4f}")
```

Exercice 03: Implémentation d'un Perceptron Multicouche (MLP) sur le jeu de données Breast Cancer Wisconsin (Diagnostic) Dataset

Le dataset Breast Cancer Wisconsin contient des informations médicales sur 569 tumeurs mammaires. Chaque instance est décrite par 30 attributs numériques (texture, compacité, concavité, etc.) et est étiquetée comme bénigne (0) ou maligne (1). C'est un problème de classification binaire.

```
data = load_breast_cancer()
X = data.data
y = data.target

# Exploration
print("Nombre d'instances :", X.shape[0])
print("Nombre d'attributs :", X.shape[1])
print("Classes :", data.target_names)
print("Distribution des classes :", np.bincount(y))

Nombre d'instances : 569
Nombre d'attributs : 30
Classes : ['malignant' 'benign']
Distribution des classes : [212 357]
```

Contrairement aux images (pixels entre 0 et 255), les attributs médicaux ont des échelles très différentes (par exemple, la texture peut varier de 9 à 40 alors que l'aire peut aller de 143 à 2501). StandardScaler centre les données autour de 0 avec un écart-type de 1, ce qui permet au modèle de converger plus rapidement.

Prétraitement des données:

```
# Séparation de l'entraînement du test
X_train_bc, X_test_bc, y_train_bc, y_test_bc = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalisation
scaler = StandardScaler()
X_train_bc = scaler.fit_transform(X_train_bc)
X_test_bc = scaler.transform(X_test_bc)

print("Taille entraînement :", X_train_bc.shape)
print("Taille test :", X_test_bc.shape)

Taille entraînement : (455, 30)
Taille test : (114, 30)
```

Construction du modèle:

```
model_bc = Sequential()

# Couche cachée 1
model_bc.add(Dense(64, activation='relu', input_shape=(30,)))

# Couche cachée 2
model_bc.add(Dense(32, activation='relu'))

# Couche de sortie
model_bc.add(Dense(1, activation='sigmoid'))

model_bc.summary()
```

Compilation et traitement:

```

Compilation du modèle

model_bc.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

Entraînement du modèle

history_bc = model_bc.fit(
    X_train_bc, y_train_bc,
    epochs=10,
    batch_size=32
)
    
```

Evaluation du modèle:

```

test_loss, test_accuracy = model_bc.evaluate(X_test_bc, y_test_bc)
print(f"\nPerte sur le test : {test_loss:.4f}")
print(f"Précision sur le test : {test_accuracy:.4f}")

4/4 ————— 0s 17ms/step - accuracy: 0.9649 - loss: 0.0739

Perte sur le test : 0.0739
Précision sur le test : 0.9649
    
```

Exercice 04: Implémentation d'un MLP pour la Prédiction du Prix d'une Maison sur le Dataset Boston Housing

Le dataset Boston Housing contient des informations sur 506 maisons situées à Boston, décrites par 13 caractéristiques (taux de criminalité, nombre de chambres, distance aux centres d'emplois, etc.). L'objectif est de prédire le prix médian des maisons en milliers de dollars. C'est un problème de régression, et non de classification.

```

(X_train_bh, y_train_bh), (X_test_bh, y_test_bh) = boston_housing.load_data()

# Exploration
print("Taille des données d'entraînement :", X_train_bh.shape)
print("Taille des données de test :", X_test_bh.shape)
print("Nombre de caractéristiques :", X_train_bh.shape[1])
print("Exemple de prix (cible) :", y_train_bh[:5])

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/boston_housing.npz
57026/57026 ————— 0s 2us/step
Taille des données d'entraînement : (404, 13)
Taille des données de test : (102, 13)
Nombre de caractéristiques : 13
Exemple de prix (cible) : [15.2 42.3 50. 21.1 17.7]
    
```

Comme pour les exercices précédents, on normalise les données avec StandardScaler car les 13 attributs ont des échelles très différentes.

```

# Normalisation des données
scaler_bh = StandardScaler()
X_train_bh = scaler_bh.fit_transform(X_train_bh)
X_test_bh = scaler_bh.transform(X_test_bh)

print("Forme des données normalisées :", X_train_bh.shape)

Forme des données normalisées : (404, 13)
    
```


Construction du modèle:

```
model_bh = Sequential()

# La 1 ère couche
model_bh.add(Dense(64, activation='relu', input_shape=(13,)))

# La 2 ème couche
model_bh.add(Dense(32, activation='relu'))

# La couche de sortie
model_bh.add(Dense(1))

model_bh.summary()
```

```
from tensorflow.keras.metrics import RootMeanSquaredError

model_bh.compile(
    optimizer='adam',
    loss='mean_squared_error',
    metrics=[RootMeanSquaredError(name='rmse')]
)
```

Entraînement

```
# Entraînement avec 50 époques, batch_size=32
history_bh = model_bh.fit(
    X_train_bh, y_train_bh,
    epochs=50,
    batch_size=32
)
```

On a pas utilisé une fonction d'activation en sortie: contrairement à Softmax ou Sigmoid, on veut prédire une valeur continue (un prix), donc la sortie est linéaire.

On a également utilisé mean_squared_error (MSE) car c'est la fonction de perte standard pour la régression. Elle mesure l'écart quadratique moyen entre les prédictions et les valeurs réelles. Et RMSE c'est la racine carrée du MSE, plus facile à interpréter car elle est dans la même unité que la variable cible (milliers de dollars).

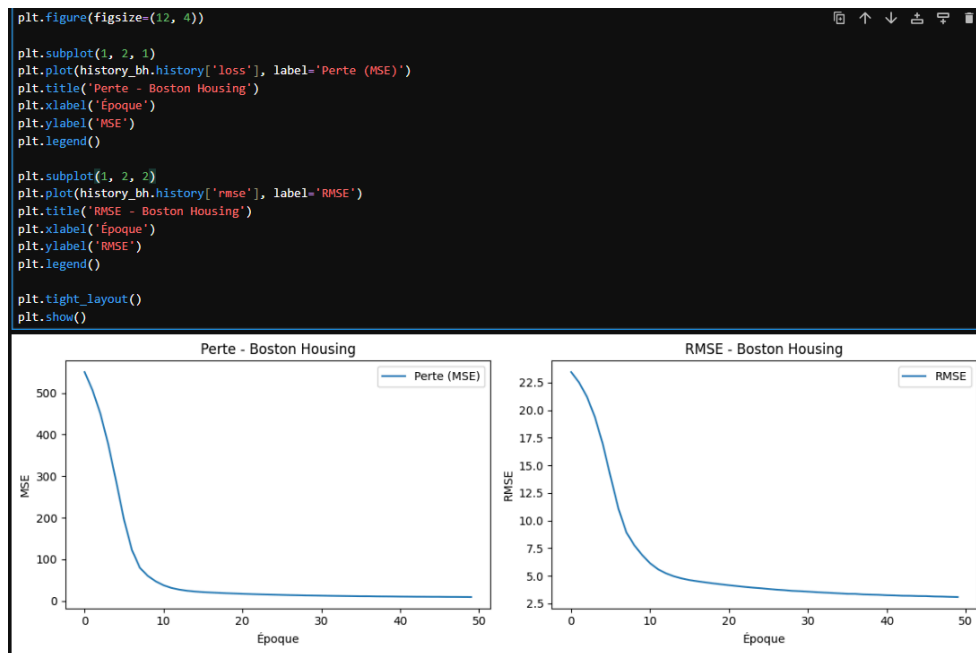
Evaluation du modèle:

```
test_loss, test_rmse = model_bh.evaluate(X_test_bh, y_test_bh)
print(f"\nPerte (MSE) sur le test : {test_loss:.4f}")
print(f"RMSE sur le test : {test_rmse:.4f}")

4/4 ————— 0s 16ms/step - loss: 23.5173 - rmse: 4.8495

Perte (MSE) sur le test : 23.5173
RMSE sur le test : 4.8495
```

Le modèle obtient un RMSE d'environ 4.84 sur le jeu de test.



Conclusion:

Ce TP nous a permis de mettre en pratique le Perceptron Multicouche (MLP) sur quatre problèmes différents, et d'en tirer plusieurs enseignements concrets. D'abord le perceptron multicouche est polyvalent, on peut l'utiliser sur différentes catégories de problèmes: classification multi-classes, classification binaire ou même la régression. Or, dans quelques cas, il montre quelques limitations au niveau de la captation des relations spatiales entre les images comme dans le cas du deuxième exercice où ces performances plafonnent autour de 50%. En somme, ce TP constitue une bonne base pour comprendre le fonctionnement du MLP et ses différentes applications.